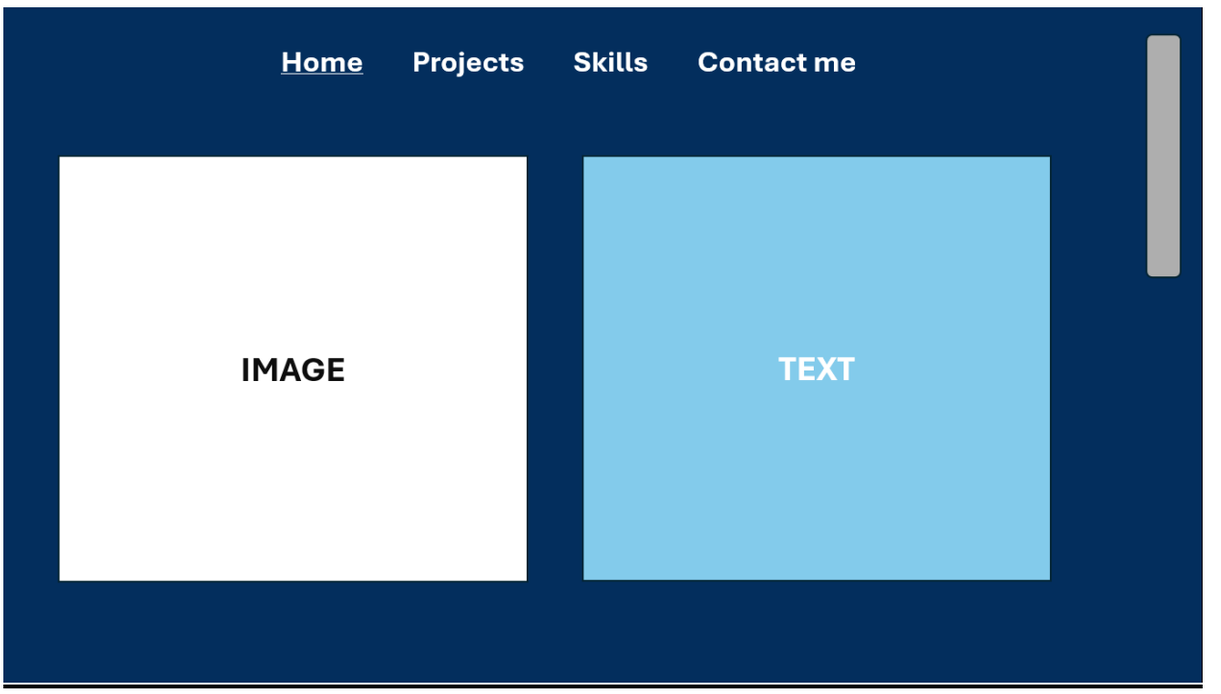
Portfolio Project documentation

Website link
http://doc.gold.ac.uk/~rmorg003/website2/index.html

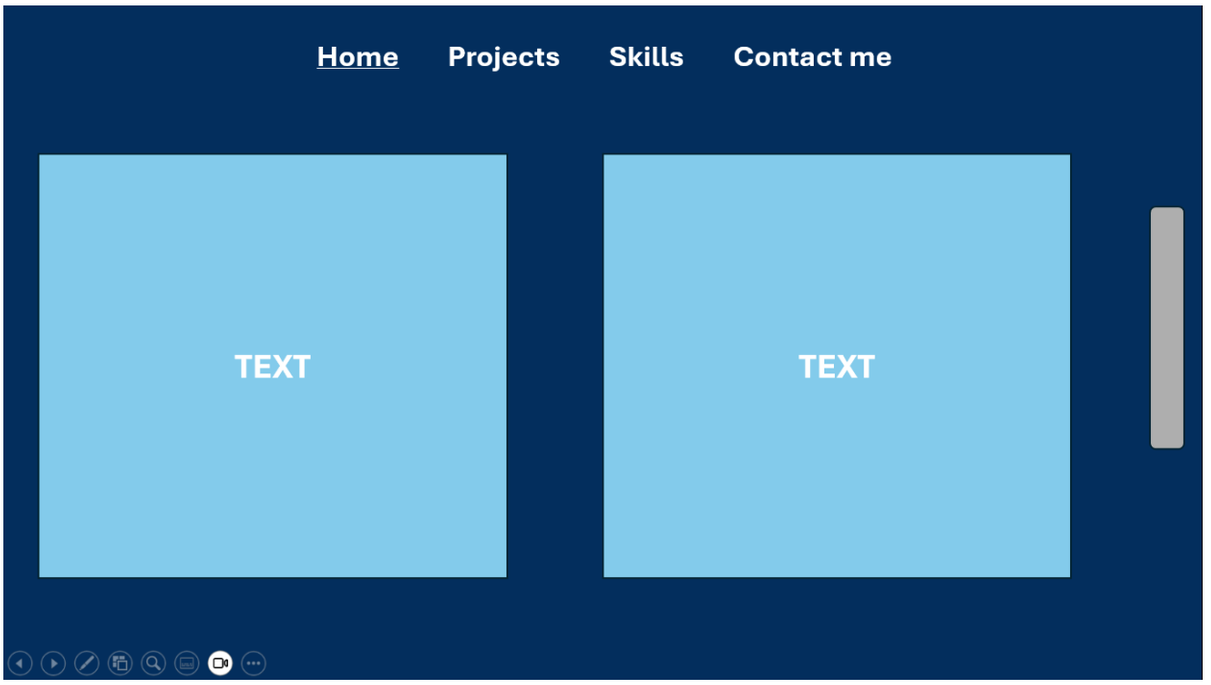
What is the purpose of my portfolio website?
The purpose of this website is to increase my employability when applying for jobs, and create an effective online presence for myself, to help me stand out from the crowd. I will also use this website to showcase my skills and projects to employers. This will hopefully enable me to get a job in the tech industry.

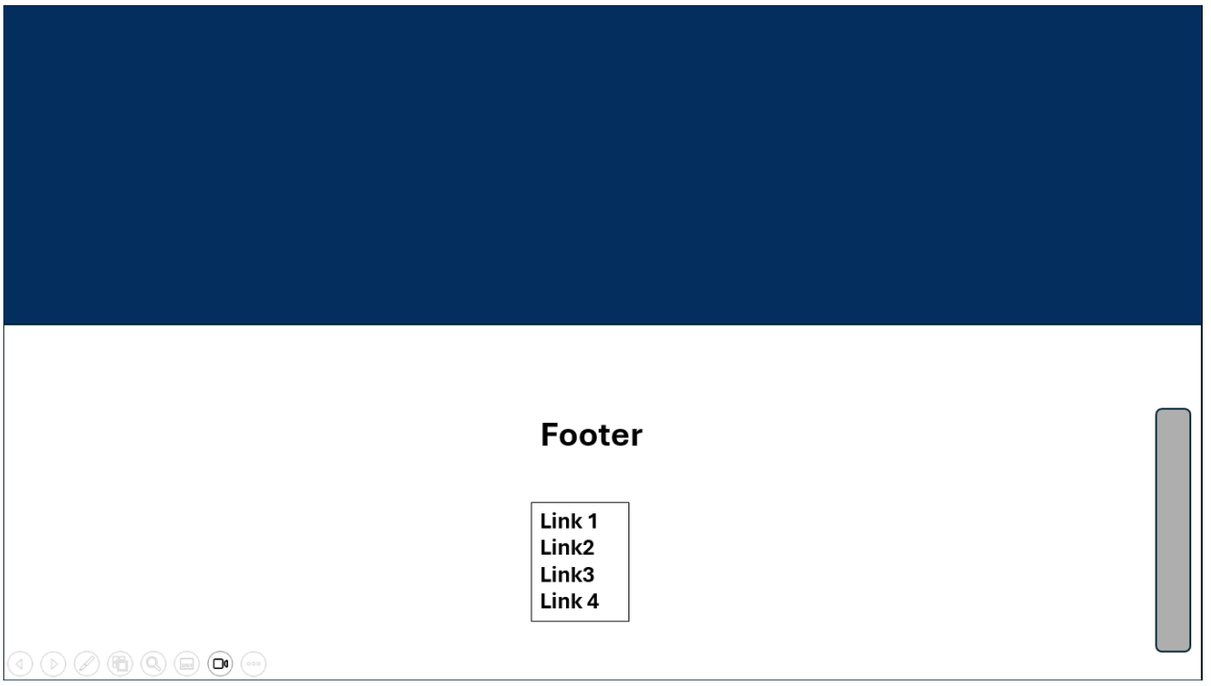
Target audience
Potential Employers/hiring managers: people who want to hire software developers into their company. On resumes they mostly look for projects and experience. Having a portfolio website helps them filter out candidates.

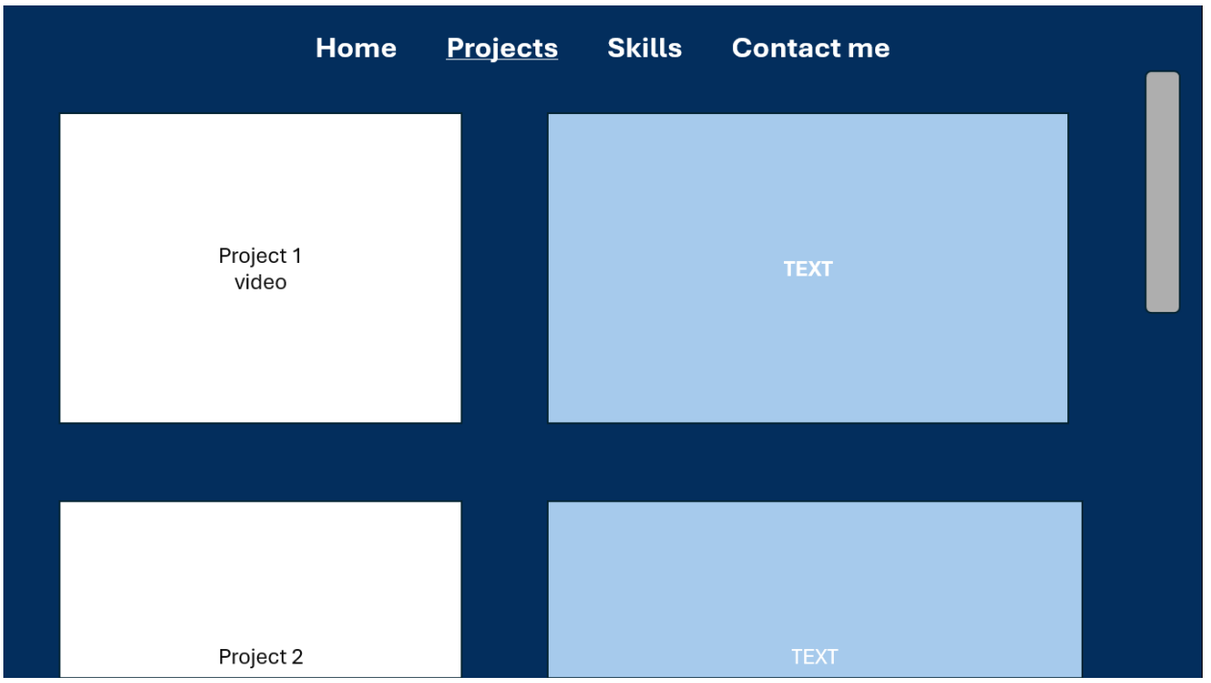
Home Page

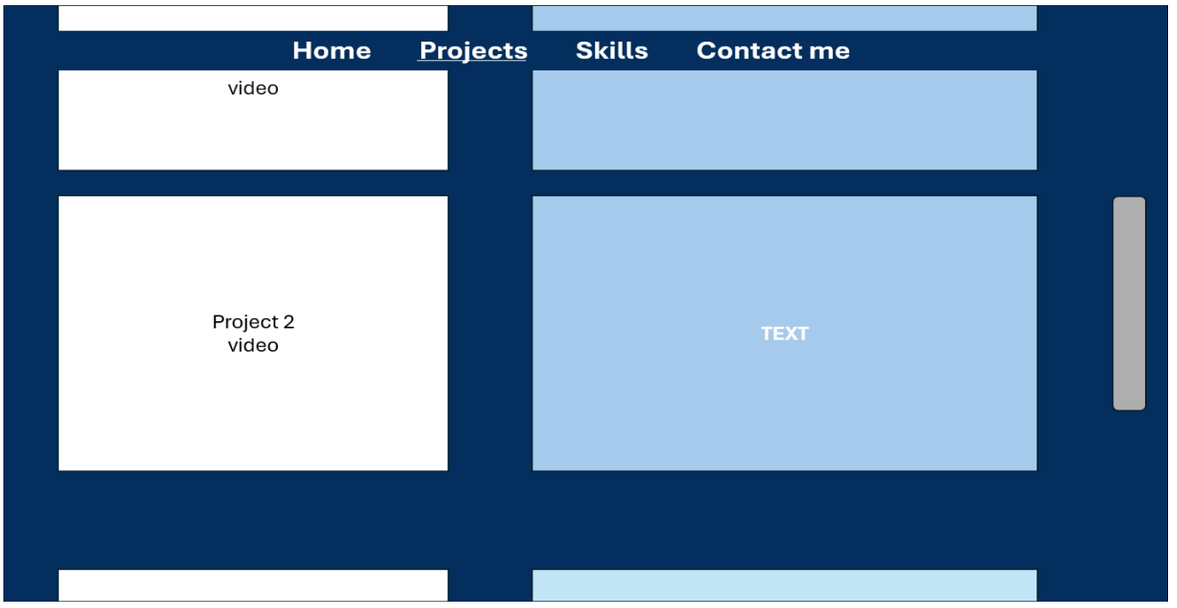


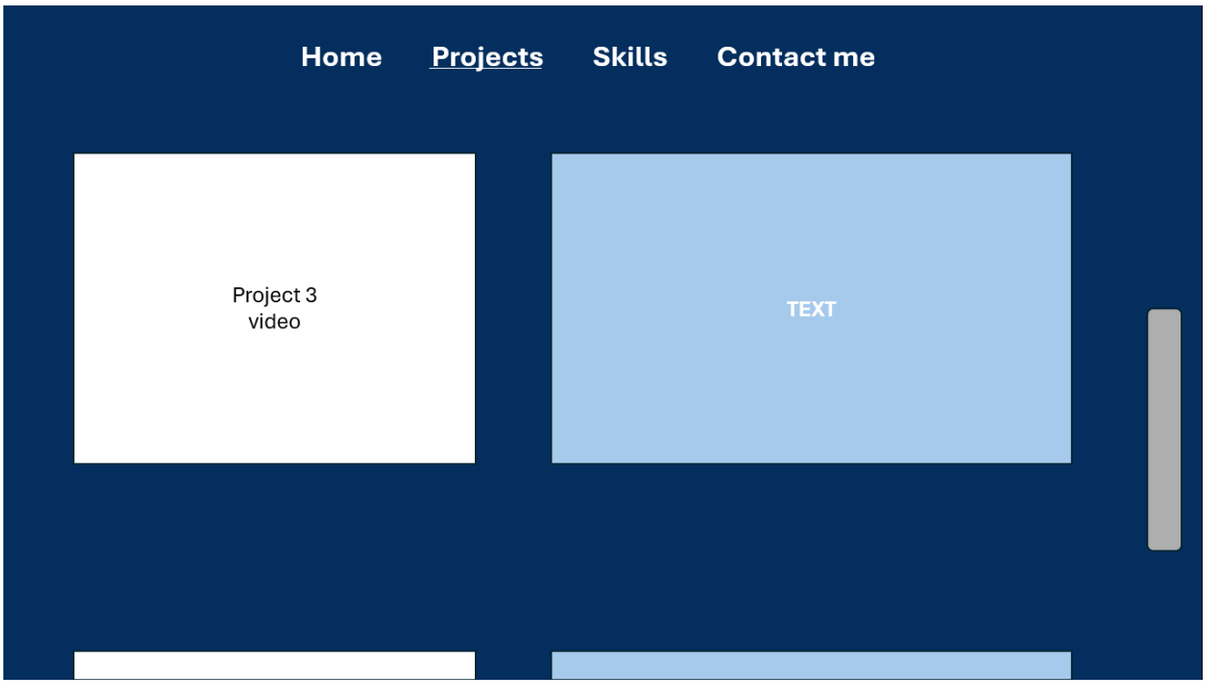
About me section (Contained within the home page for easy accessibility)

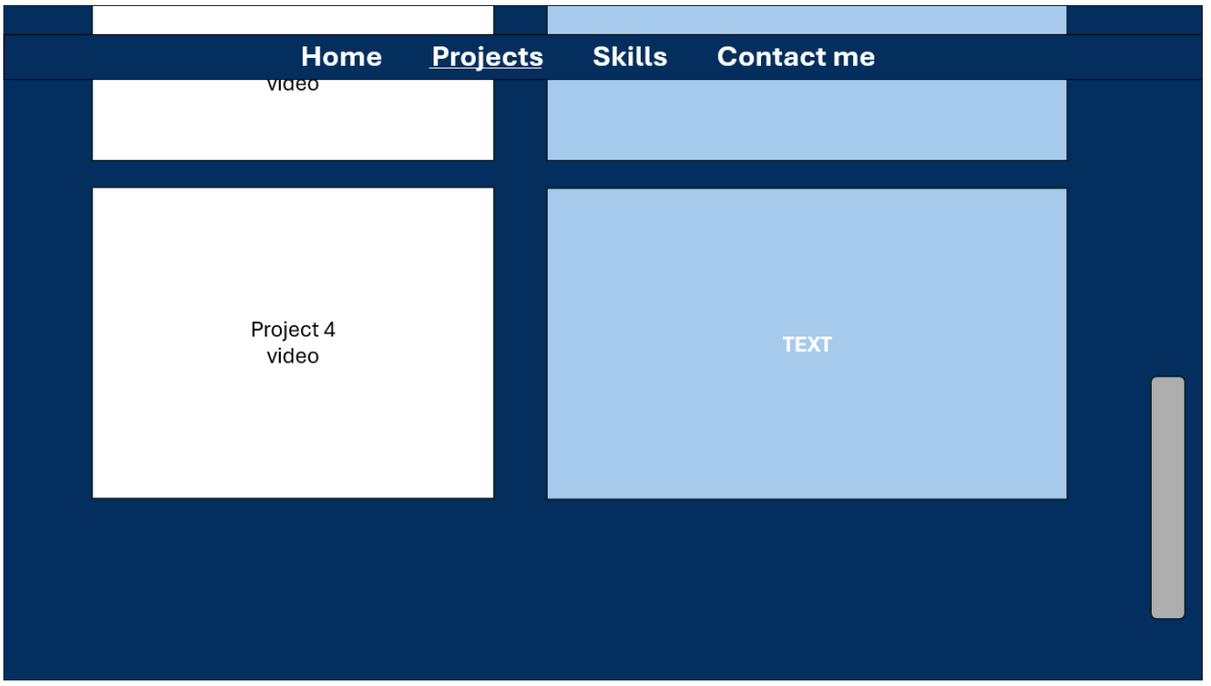

Home
Projects
Skills
Contact me
TEXT
TEXT

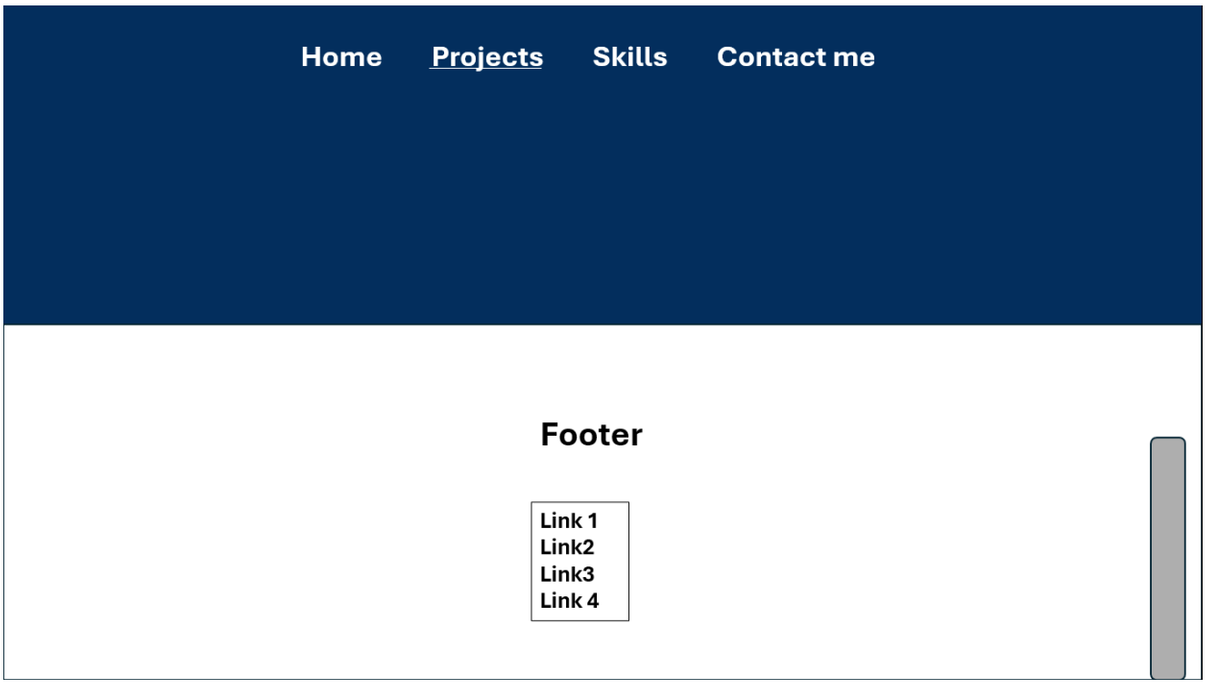

Footer
Link 1
Link2
Link3
Link 4


Home
Projects
Skills
Contact me
Project 1
video
TEXT
Project 2
TEXT

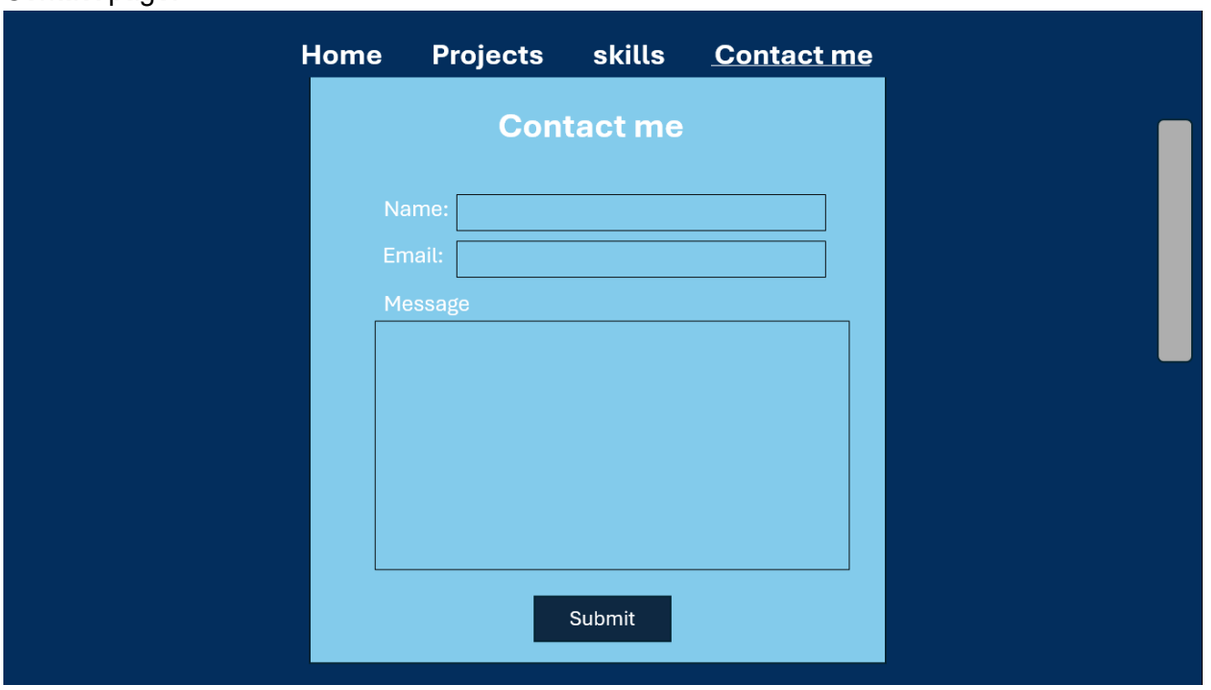

Home
Projects
Skills
Contact me
video
Project 2
video
TEXT

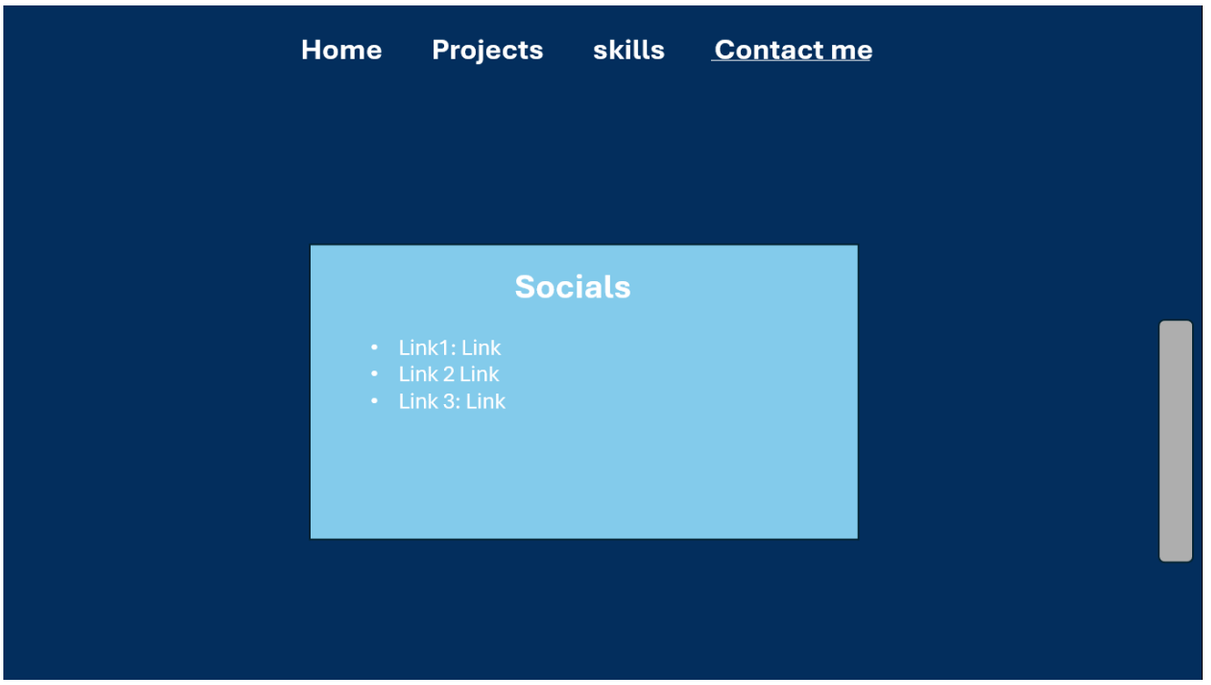

Home
Projects
Skills
Contact me
Project 3
video
TEXT


Home
Projects
Skills
Contact me
video
Project 4
video
TEXT

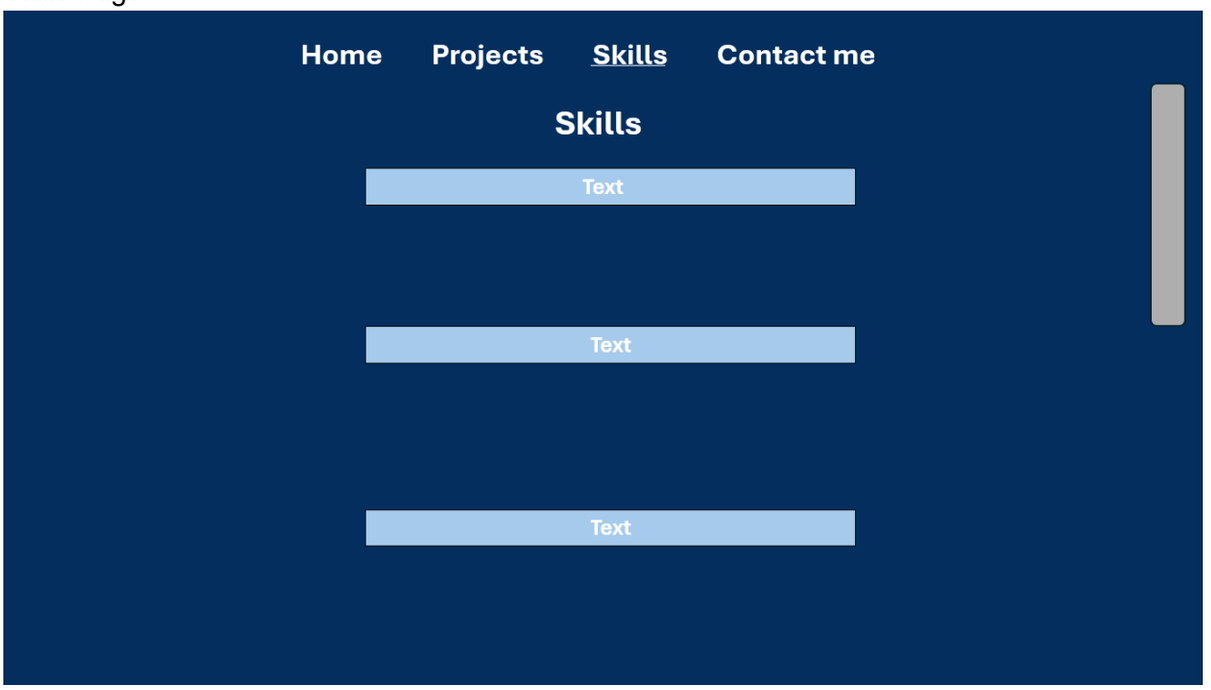


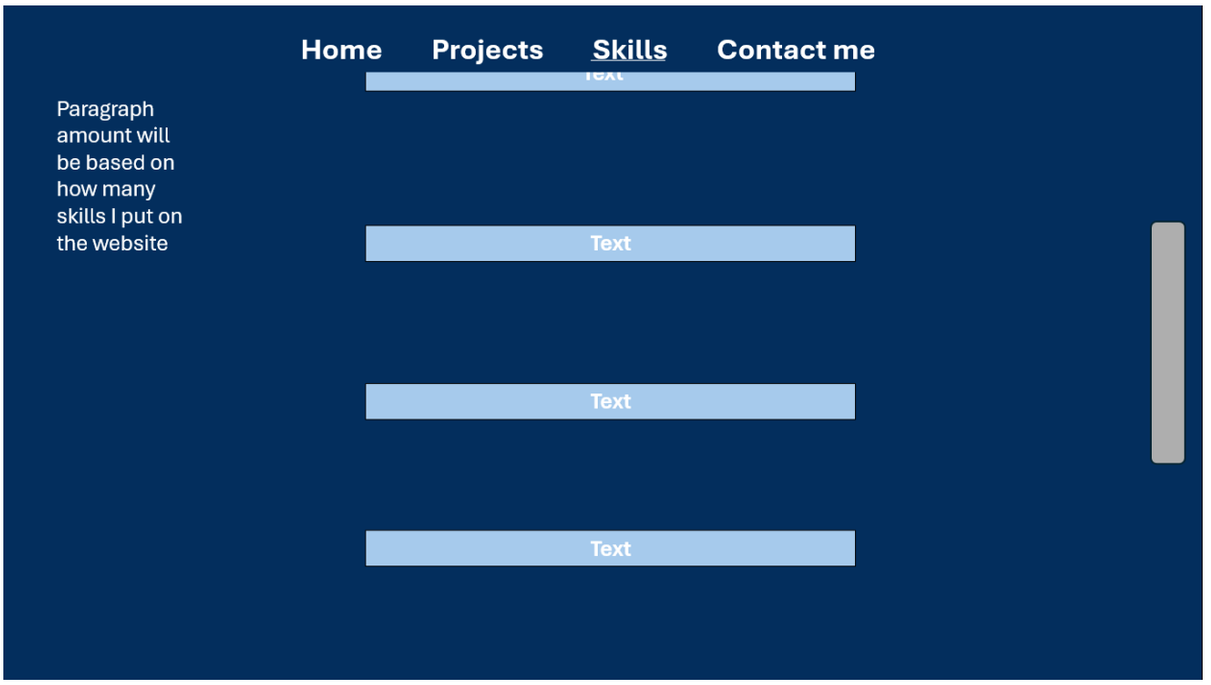
Contact page.

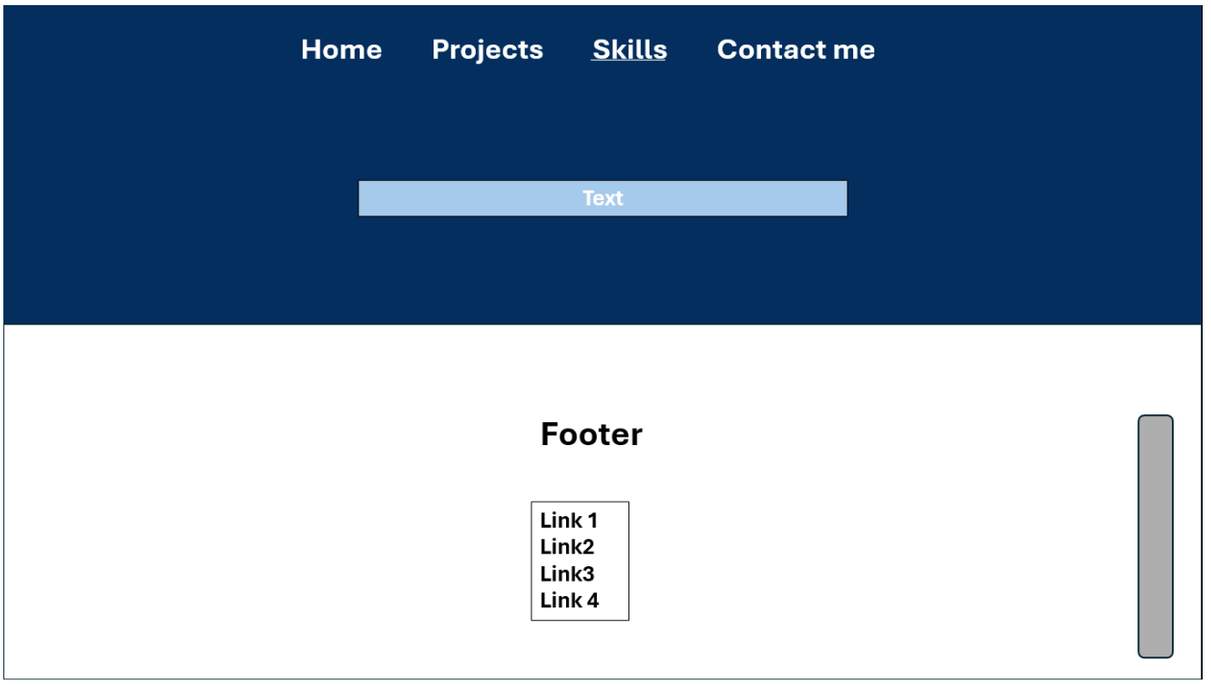



Home
Projects
skills
Contact me
Socials
Link1: Link
Link 2 Link
Link 3: Link

Skills Page


Home
Projects
Skills
Contact me
Skills
Text
Text
Text





Mockup rationale

Colour scheme

For the colour scheme, I have chosen dark blue and light blue. Since these colours are the same but in different shades, I thought they would complement each other. Adding blue to your website also gives the user peace of mind and makes them calm and relaxed. However, this decision is purely experimental and is subject to change in the future.

Layout

For the layout I used boxes that either go horizontally across the page or vertically down the page. These boxes will contain distinct types of data such as images, text, and videos.

I have used this simple layout because it is easy to implement and is consistent and responsive on different screens. Furthermore, this layout will allow me to write less code and is flexible.

Navigation



For the navigation I just have the links centered at the very top of the page. I have done it this way so that the user will easily be able to see all the different pages on my site. I have also made the navigation horizontal for this exact reason. Horizontal navigation can be easily. adapted for smaller screens as you can simply use flex: wrap to wrap the navbar around itself. Applying this to a vertical navbar would be more tedious and not yield the same result. Moreover, horizontal navbars are common for many popular websites like Microsoft, Coursera and W3schools.

Secondly, my website will have less pages than most so naturally centering four links at the top of the page would make the page look more uniform.

Structure
The navigation serves a dual purpose. One being to categorize information on separate pages and two which is to show order of importance. The Home page is first as it is arguably the most important page as it gives a first impression to the employer. If the employers like the home page, chances are that they will be excited to click onto the proceeding pages.

HTML structure and semantic elements

For the html structure, I used div elements to separate content into sections, chunks.
I attached classes and Id to these divs to then target them using CSS. I repeated this structure for every page. For example,

```
<div class="hero-card" id="toggle_colour1">
    <div id="image-card">
    <img src="images/pic of me.jpg" class="my_image"
alt="Image of myself" width="500" height="500">

</div>
```

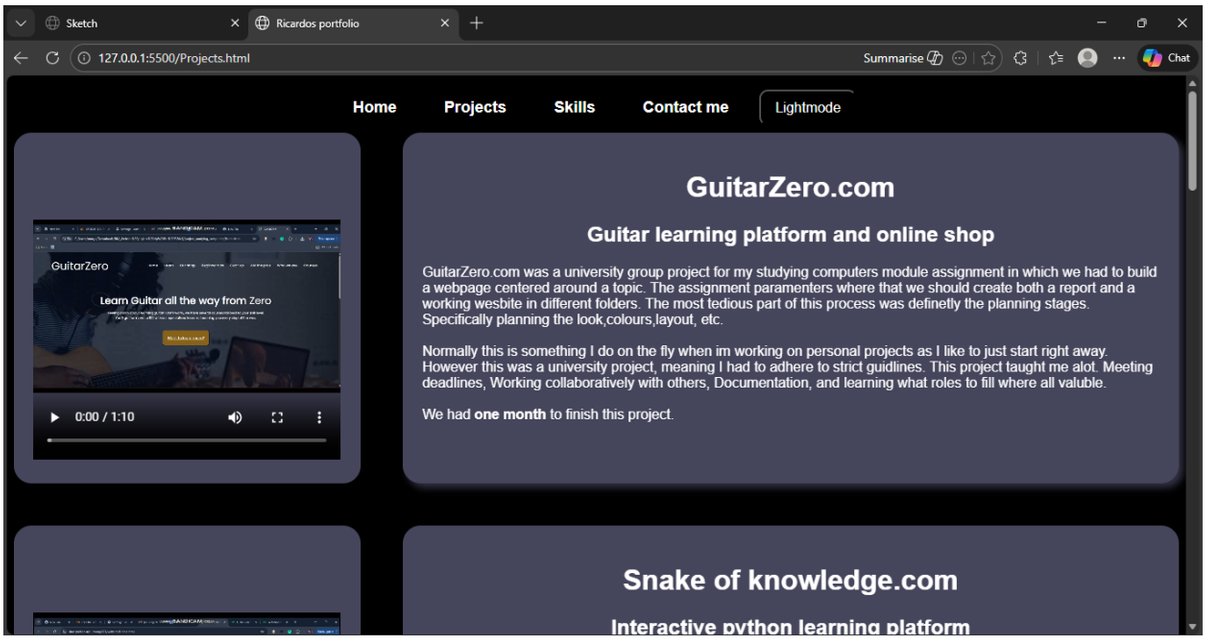
Nesting divs inside of each other meant that I could style all the children of the parent div at once, which would in turn make the CSS much easier manipulated.

Header tags
Header tags such as H1, H2, H3 etc were all used throughout my webpage. This was to show Hierarchy and bring emphasis to certain words. For example, Ricardo Morgan, About me etc. This is important as larger words are easier to see, and your eyes are more likely to gravitate towards them first. Depending on where I want the user to focus, I will make the headings larger/smaller as I see fit.

<nav> tag

The nav element was used to define the navigation links of the webpage. By default, it aligned the html link horizontally. This saved me a lot of time when creating the layout and spacing for the navbar.

<Video> tag
Used to add videos onto my website. This tag can also be used to change the width and height of the video, add autoplay and many other features. I nested several video tags into dividers to display them one by one. I then nested all those dividers into a parent div to display all the videos and text in a grid like pattern. Like you see here.



 Element
I used an unordered list element to make a list of my skills as bullet points. This was used to quickly show a list of my skills on the home page.

<p> element
Paragraph tags were used to write paragraphs on all pages. I used this element to write out. Explanations of my projects, the about me section and more.

<image> tag
I used one image tag on the home page of the website (called index.html). This was so I could add an image of myself onto it. Developers having images of themselves on the landing page is widespread practice on most portfolio websites.

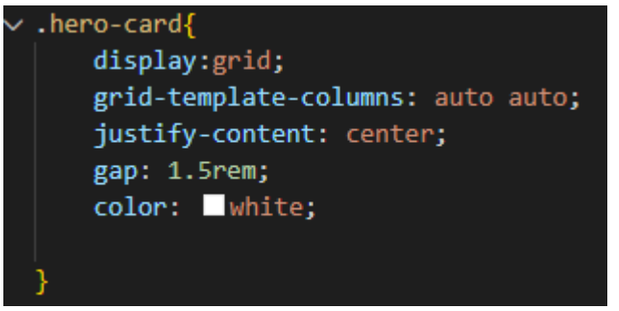
 tag
To quickly align specific text so the user has an easier time focusing on the headings.

<footer> tag
I used the footer element to create a footer at the bottom of my page. This widespread practice in web development for accessibility/ease of use reasons. This footer contains the links that are in the nav element.

CSS design choices

Grid

Using the grid layout module, I split the content of my pages into columns and rows. Using this technique allowed me to easily create a responsive layout structure and allows for added complexity.



I specify the size of each column and how many columns there are using the CSS grid property grid-template-columns. I set them both to default (auto), so my markup is consistent. I also decided to have a gap of around 1.5xrem. Since rem is a relative unit (Relative to the font-size of the root HTML element) it scales better for different screen sizes rather than absolute units. That is why I used rem on the gap grid property. The class hero-card is my parent container for this layout. This makes it so all the content appear in a column row format with the child divs arranged one next to the other.

CSS transitions and transformations

I first added a transition(animation) to my navigation bar. I did this because I wanted a smooth hover effect when the user hovers over a nav link. I accomplished this by doing the following.

- Find the parent class of the nav-bar.
- Reference all dividers within the nav element
- Choose what CSS property to change.
- Choose transition duration, for example three seconds
- Choose transition start up time like ease-in
- Add the property to the same class but referencing the hover.

I added the transformation property to my image. I did this by scaling down and skewing the image at the same time on hover. Using both transitions and transformations created this significant effect as you can see. The effect scales down the image by 1.5px and skews the Y axis at an angle.



I also slowly transition the background color of the div from dark blue to lightblue. This is also thanks to transformations.

Box shadows

I use the box shadow property and transition property together, on all my dividers and my image. This is to create a smooth clean 3d effect whilst maintaining clean code.

Flexbox

I used the flexbox Layout model to display items horizontally within a container. The parent container being the navigation bar class named nav-bar. I used flexbox for the nav specifically because it can either be used for horizontal or vertical alignment of items. Moreover, flexboxes are responsive and will adapt to almost any screen size. To make the navbar even more responsive, I used a media query to change the flex-wrap to wrap when the viewport size reached a minimum width of 300px.

@media queries accessibility and responsiveness

I used this rule to apply assorted styles to elements based on screen size. I used this rule to make my website change styles dynamically, which ensures responsiveness on all types of screen sizes. I have used media queries on several elements on my website to change the styling based on screen size. I have also changed other areas of my webpage such as Video tag width, project div columns and more.

```
@media only screen and (min-width: 300px){
    .project_card{
        grid-template-columns: auto;
    }
    video{
        width: 100%;
        aspect-ratio: 1;
    }

    .nav-bar{
        flex-wrap: wrap;
    }
```

Using media queries made the website more accessible to users as they can look at the website comfortably on any device.

Typography
I use font arial because it is one of the more common font types used amongst websites.
It is very clear and readable. Most people read/write in arial so many are used to it naturally.

JavaScript functionality

Light mode/dark mode toggle
To create this background, I changed the background of my website to black. This is so that light mode would be easier to implement as it directly contrasts with black. I use js function toggle so I can add and remove the light mode class from the home page ID. This is displayed below.

```
function lmtoggle(){
    document.body.classList.toggle("light-mode")
    let btn = document.getElementById("btn");

    if (btn.innerHTML === "Lightmode"){
        btn.innerHTML = "Darkmode";
    } else {
        btn.innerHTML = "Lightmode"
    };

    var home = document.getElementById("text_colour_change");
    home.classList.toggle("text_colour");
```

The function is called once the user presses the button. This is what the onclick event is for.
I add the onclick event inside the button element, calling the function within the event so it is called every click.

Moreover, I change the buttons html so that depending on what the text is, the inner html will change to “Dark mode” else it will be Light mode. Finally, I store an element based on its ID into a variable (In this case the id of the home page parent div) to add and remove the text colour class. Adding this class puts the styling inside that class onto the parent div therefore applying the styling to every child of the parent div.

Color changes for Js functionality.
Originally, I planned for my website to have a dark blue and light-blue color scheme. However, when creating the dark mode/ light mode feature, I found this difficult to implement as my colour scheme was dark blue/light blue instead of black and white. To make this easier on myself I decided to go with a black and white colour scheme as it is easier on the eyes. I keep the divider colour a pale darkish blue as to make the website more exciting to look at instead of just being black and white only.

File Management

I put all my files into a single folder. This was so I could open the folder into visual studio without having to open every individual file. Since this is an independent project, I don’t have to use

version control resources such as Git. I also stored my images and videos in separate folders. These folders would then go into the main folder with everything else (Called Portfolio project). Doing It this way made everything easier to manage as it was all in the same directory.

Reflection

What worked well?

Using the Grid layout module works tremendously well. This allowed me to save so much time styling elements as I did not have to worry about responsiveness as much (Grid layout is naturally responsive to different screen sizes). Using flexbox was great for the navbar as I didn't need to use columns or margins for spacing. Media queries were an immense help in making my website more accessible for different devices as it allowed the styling to adapt based on screen size.

Challenges faced

I faced many challenges in the development phase of my website. When I created the mockup, I had to change the layout 3 separate times. I finally settled on a box layout for the final design, as I knew it would be straightforward to implement into code. I had difficulty figuring out appropriate class names for html markup. I eventually went with basic names such as hero or project. Using many divider tags was tedious and time consuming, but it paid off in the end.

Potential Improvements

- Adding a better theme toggle instead of being a button with text on it
- Having a concreate colour scheme (Improved planning)
- Making the text go black when on light mode for all pages.
- Using a slideshow instead of videos in order to decrease load times and bandwidth
- Decreasing video length and increasing its speed
- Adding a function that autoplay's videos on hover and stops video off hover.
- Changing the colour scheme to make the website more unique
- Adding a hamburger menu using media queries to increase accessibility and useability
- Improve paragraph content by adding more detail.
- Adding all the content on a single page instead of segmenting